Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 10

Disciplina: Tehnici de programare

Tema: ” Funcții și excepții”

A efectuat : st. gr.SI – 263 Morozovschii Carolina

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Capitolul 1. Extragerea unui eșec dintr-o linie

Sarcina: Să definesc o expresie regulată și funcția esec(linie). Funcția întoarce un tuplu (user, ip) dacă linia indică o autentificare eșuată, iar în caz contrar întoarce None.

```python
import argparse
import re
from collections import Counter

TIPAR = re.compile(r"Failed password for (?:invalid user )?(\S+) from (\S+) port")


def esec(linie):
    m = TIPAR.search(linie)
    if m is None:
        return None
    return (m.group(1), m.group(2))
```

Fig.1.1-Codul sursă (parse_auth.py)

```
PS C:\Users\moroz\Documents\Lucrari-de-laborator> python -m arsenal.aparare.parse_auth
('root', '203.0.113.9')
None
```

Fig.1.2-Rezultatul codului (parse_auth.py)

Capitolul 2. Citirea jurnalului și tratarea liniilor stricate

Sarcina: Să citesc fișierul linie cu linie, să ignor liniile care nu sunt eșecuri și să tratez cazul în care fișierul nu există.

```python
def citeste_esecuri(cale):
    esecuri = []
    try:
        f = open(cale, encoding="utf-8")
    except FileNotFoundError:
        print("nu gasesc", cale)
        return esecuri

    with f:
        for linie in f:
            r = esec(linie)
            if r is not None:
                esecuri.append({"user": r[0], "ip": r[1]})
    return esecuri
```

Fig.2.1- Codul sursă (parse_auth.py)

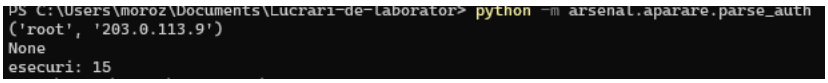


Fig.2.2-Rezultatul codului (parse_auth.py)

Capitolul 3. Numărarea eșecurilor pe adresă IP

Sarcina: Să număr de câte ori apare fiecare adresă IP, folosind Counter.

```
def pe_ip(esecuri):
    return Counter(e["ip"] for e in esecuri)
```

Fig.3.1- Codul sursă (parse_auth.py)

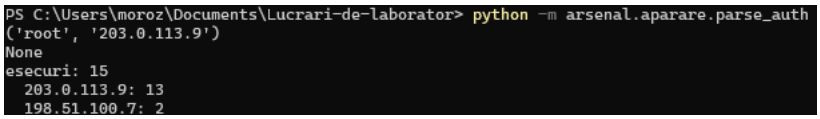


Fig.3.2-Rezultatul codului (parse_auth.py)

Capitolul 4. Funcția principală

Sarcina: Să leg citirea jurnalului de numărarea pe IP și să afișez rezumatul.

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("log")
    args = parser.parse_args()

    esecuri = citeste_esecuri(args.log)
    counts = pe_ip(esecuri)
    print("Esecuri de autentificare:", len(esecuri))
    for ip, n in counts.most_common():
        print(f"  {ip}: {n}")


if __name__ == "__main__":
    main()
```

Fig.4.1- Codul sursă (parse_auth.py)

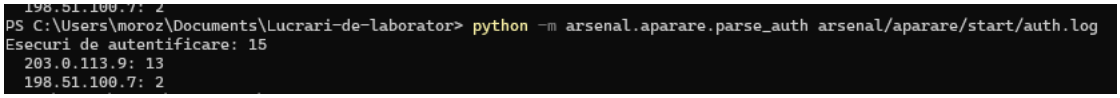


Fig.4.2-Rezultatul codului (parse_auth.py)

Capitolul 5. Funcția suspecti() și pragul implicit

Sarcina: Să întorc IP-urile cu cel puțin numărul dat de eșecuri, sortate descrescător. Pragul implicit este 10.

```python
import argparse
import json
from collections import Counter

from arsenal.aparare.parse_auth import citeste_esecuri


def suspecti(counts, prag=10):
    if prag < 1:
        raise ValueError(f"prag invalid: {prag} (cel putin 1)")
    return sorted(
        [(ip, n) for ip, n in counts.items() if n >= prag],
        key=lambda element: element[1],
        reverse=True,
    )
```

Fig.5.1- Codul sursă (detector.py)

```
PS C:\Users\moroz\Documents\Lucrari-de-laborator> python -m arsenal.aparare.detector
[('203.0.113.9', 13)]
[('203.0.113.9', 13), ('198.51.100.7', 2)]
```

Fig.5.2-Rezultatul codului (detector.py)

Capitolul 6. Validarea pragului

Sarcina**:** Să opresc funcția și să afișez o eroare clară dacă pragul este mai mic decât 1.

```
PS C:\Users\moroz\Documents\Lucrari-de-laborator> python -c "from arsenal.aparare.detector import suspecti; suspecti({}, 0)"
```

Fig.6.1-Comanda de verificare

```
  File "C:\Users\moroz\Documents\Lucrari-de-laborator\arsenal\aparare\detector.py", line 6, in suspecti
    raise ValueError(f"prag invalid: {prag} (cel putin 1)")
ValueError: prag invalid: 0 (cel putin 1)
```

Fig.6.2-Rezultatul afisat

Capitolul 7. Argumente în linia de comandă și raport JSON

Sarcina: Să accept calea jurnalului, un --prag opțional și comutatorul --json. Fără --json, programul afișează un rezumat. Cu --json, afișează rezultatele ca JSON.

```python
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("log")
    parser.add_argument("--prag", type=int, default=10)
    parser.add_argument("--json", action="store_true")
    args = parser.parse_args()

    esecuri = citeste_esecuri(args.log)
    counts = Counter(e["ip"] for e in esecuri)
    sus = suspecti(counts, args.prag)

    if args.json:
        raport = {
            "esecuri": esecuri,
            "pe_ip": counts,
            "suspecti": sus,
        }
        print(json.dumps(raport, indent=2, ensure_ascii=False))
        return

    print("Esecuri de autentificare:", len(esecuri))
    for ip, n in counts.most_common():
        print(f"  {ip}: {n}")
    print(f"Suspecti (prag {args.prag}):")
    for ip, n in sus:
        print(f"  {ip}  ({n} incercari)  BRUTE FORCE")


if __name__ == "__main__":
    main()
```

Fig.7.1- Codul sursă (detector.py)

```
PS C:\Users\moroz\Documents\Lucrari-de-laborator> python -m arsenal.aparare.detector arsenal/aparare/start/auth.log
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
Suspecti (prag 10):
  203.0.113.9  (13 incercari)  BRUTE FORCE
```

Fig.7.2-Rezultatul codului (detector.py)

Capitolul 8. Teste automate

Sarcina: Să verific cu date scrise direct în test că pragul filtrează corect IP-urile și că un prag invalid ridică ValueError.

```python
from collections import Counter

import pytest

from arsenal.aparare.detector import suspecti


def test_pragul_taie():
    counts = Counter({"a": 13, "b": 2})
    assert suspecti(counts, 10) == [("a", 13)]


def test_prag_invalid():
    with pytest.raises(ValueError):
        suspecti({}, 0)
```

Fig.8.1- Codul sursă (test_detector.py)

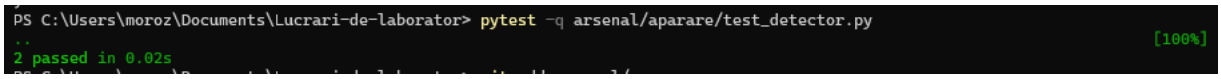


Fig.8.2-Rezultatul codului (test_detector.py)

CONCLUZII

Lucrarea folosește funcții care întorc rezultate, o expresie regulată pentru extragerea datelor, try/except pentru tratarea unui fișier lipsă, Counter pentru numărarea IP-urilor și raise ValueError pentru validarea pragului. Jurnalul conține 15 eșecuri: 13 asociate adresei 203.0.113.9 și 2 asociate adresei 198.51.100.7. Cu pragul implicit 10, prima adresă este semnalată ca suspectă de brute force.

BIBLIOGRAFIE

1. Github
2. Windows Powershell